

Fișa de lucru

Tablouri unidimensionale (Liste)

Declarare | Accesare | Parcurgere | Operații | Sortare

Competențe vizate

- Declararea și inițializarea listelor în Python.
- Accesarea și modificarea elementelor prin index.
- Parcurgerea listelor și aplicarea operațiilor elementare.
- Implementarea algoritmilor de căutare și sortare.

1. Noțiuni teoretice

1.1 Declararea și structura unei liste

Lista (tablou unidimensional)

O listă este o structură de date care reține o colecție ordonată de elemente accesibile prin index numeric. În Python, o listă poate conține elemente de orice tip. Indexarea începe de la 0 (primul element are indexul 0, al doilea are indexul 1, ultimul are indexul n-1).

```
# Declarare și inițializare:  
v = [10, 20, 30, 40, 50]      # listă cu 5 elemente întregi  
nume = ['Ana', 'Ion', 'Maria'] # listă de șiruri de caractere  
v_gol = []                   # listă vidă (goală)  
  
# Accesarea elementelor:  
print(v[0])    # afișează 10 (primul element)  
print(v[4])    # afișează 50 (ultimul element)  
print(v[-1])   # afișează 50 (ultimul, indexare negativă)  
v[2] = 99      # modifică elementul de pe poziția 2
```

1.2 Funcții și metode esențiale

Funcție / Metodă	Ce returnează / face	Exemplu
len(lista)	Numărul de elemente	len([3,1,4,1]) → 4
sum(lista)	Suma elementelor (doar numere)	sum([3,1,4]) → 8
max(lista)	Elementul maxim	max([3,1,4]) → 4
min(lista)	Elementul minim	min([3,1,4]) → 1
lista.append(x)	Adaugă x la sfârșit (in-place)	v.append(7)
lista.insert(i, x)	Inserează x pe poziția i (in-place)	v.insert(0, 99)
lista.remove(x)	Șterge prima apariție a valorii x	v.remove(20)
del lista[i]	Șterge elementul de pe poziția i	del v[2]
lista.sort()	Sortează lista crescător (in-place)	v.sort()
sorted(lista)	Returnează o copie sortată (lista originală neschimbată)	s = sorted(v)
x in lista	True dacă x se află în listă	5 in v → False

1.3 Parcurgerea listelor

Parcurgerea (iterarea) unei liste se realizează cu bucla `for`. Există două stiluri principale:

```
v = [10, 20, 30, 40, 50]

# Stil 1: parcurgere prin valori
for element in v:      # element ia pe rând fiecare valoare
    print(element)      # afișează 10, 20, 30, 40, 50

# Stil 2: parcurgere prin index
for i in range(len(v)): # i ia valorile 0, 1, 2, 3, 4
    print(i, v[i])      # afișează indexul și valoarea
```

1.4 Citirea unei liste de la tastatură

```
n = int(input('Numărul de elemente: '))
v = []
for i in range(n):
    x = int(input('v[' + str(i) + '] = '))
    v.append(x)
print('Lista citită:', v)
```

2. Exemple rezolvate

Exemplul 1 — Calculul sumei, mediei, minimului și maximului unei liste

Se citește un șir de n numere întregi într-o listă. Se calculează și se afișează suma, media aritmetică, elementul minim și cel maxim.

```
n = int(input('n = '))
v = []
for i in range(n):
    x = int(input('Element: '))
    v.append(x)

# Calcule folosind funcțiile Python
suma = sum(v)
medie = suma / n
minim = min(v)
maxim = max(v)
print('Suma: ', suma)
print('Media: ', round(medie, 2))
print('Minim: ', minim)
print('Maxim: ', maxim)
```

Exemplu de execuție (n=5, v=[3, 7, 1, 9, 2]):

```
Suma: 22
Media: 4.4
Minim: 1
Maxim: 9
```

Exemplul 2 — Căutarea secvențială într-o listă

Se citește o listă și o valoare x . Se verifică dacă x se află în listă și, dacă da, se afișează poziția primei apariții.

```

n = int(input('n = '))
v = []
for i in range(n):
    v.append(int(input('Element: ')))
x = int(input('Valoarea căutată: '))

# Căutarea secvențială
găsit = False
for i in range(n):
    if v[i] == x:
        print('Valoarea', x, 'găsită pe poziția', i)
        găsit = True
        break
if not găsit:
    print('Valoarea', x, 'nu se află în listă.')

```

Exemplu de execuție (v=[3,7,1,9,2], x=9):

Valoarea 9 găsită pe poziția 3

Exemplul 3 — Sortarea unei liste prin selecția minimului

Se citește o listă și se sortează crescător folosind algoritmul de sortare prin selecția minimului.

```

n = int(input('n = '))
v = []
for i in range(n):
    v.append(int(input('Element: ')))

# Sortare prin selecția minimului
for i in range(n - 1):
    # Găsim poziția minimului din subșirul v[i..n-1]
    poz_min = i
    for j in range(i + 1, n):
        if v[j] < v[poz_min]:
            poz_min = j
    # Interschimbăm v[i] cu minimul găsit
    v[i], v[poz_min] = v[poz_min], v[i]
print('Lista sortată:', v)

```

Exemplu de execuție (v=[5,2,8,1,4]):

Lista sortată: [1, 2, 4, 5, 8]

3. Exerciții

- 1 **Ușor** Citiți o listă de n numere întregi și afișați: (a) elementele de pe poziții pare (0, 2, 4, ...); (b) elementele de pe poziții impare (1, 3, 5, ...); (c) lista în ordine inversă.
- 2 **Ușor** Citiți o listă de n numere întregi și numărați câte elemente sunt: pozitive, negative și egale cu zero.
- 3 **Mediu** Citiți două liste de aceeași lungime cu note ale unui elev la două materii diferite. Afișați o a treia listă care conține suma notelor de pe fiecare poziție.
 - a) Citiți n, apoi cele n elemente ale primei liste.
 - b) Citiți cele n elemente ale celei de-a doua liste.
 - c) Construiți lista sumelor și afișați-o.

- 4 **Mediu** Citiți o listă cu n numere reale. Înlocuiți fiecare element negativ cu valoarea sa absolută și afișați lista modificată. Afișați și media elementelor din lista finală.

Indiciu: valoarea absolută a unui număr x se obține cu `abs(x)` sau condiția: dacă $x < 0$ atunci $x = -x$.

- 5 **Difil** Citiți o listă de n numere întregi. Ștergeți toate aparițiile valorii x (citită de la tastatură) din listă. Afișați lista rezultată și lungimea sa.

Metoda recomandată: construiți o listă nouă cu elementele diferite de x .

Alternativă: utilizați metoda `.remove()` într-o buclă `while`.

- 6 **Difil** Citiți două liste sortate crescător de lungimi m și n . Interclasați-le într-o a treia listă sortată, fără a folosi `sorted()`. Afișați lista rezultată.

Indiciu: folosiți doi indici i și j care parcurg simultan cele două liste.

La fiecare pas, adăugați în lista rezultat minimul dintre `v1[i]` și `v2[j]`.

La final, adăugați elementele rămase din lista care nu s-a epuizat.

Verifică ce ai învățat

1. Care este indexul primului element al unei liste în Python? Dar al ultimului element, pentru o listă cu n elemente?
2. Care este diferența dintre `lista.sort()` și `sorted(lista)`? În ce situație este recomandată fiecare variantă?
3. Explicați algoritmul de căutare secvențială. Câte comparații sunt necesare în cel mai nefavorabil caz, pentru o listă cu n elemente?
4. Cum se adaugă un element la sfârșitul unei liste? Dar la începutul listei? Dați exemple de cod.
5. Scrieți un program care citește o listă de n numere și verifică dacă este sortat crescător, afișând un mesaj corespunzător.
6. Descrieți algoritmul de sortare prin selecția minimului. Câte perechi de interschimbări se fac în cel mai nefavorabil caz, pentru o listă cu n elemente?